Cognizant Digital Nurture 5.0

ReactJS Hands-On Exercise 14

React Context API

Objective

The objective of this exercise is to understand and implement the **React Context API** to avoid prop drilling. The application demonstrates how data can be shared directly between components without passing props through every intermediate component. In this exercise, a ThemeContext is created to share the theme value across nested components using the useContext() hook. The provided employee management application is modified to consume the context instead of receiving the theme as a prop.

Theory

What is React Context API?

React Context API is a built-in feature that allows data to be shared across multiple components without manually passing props at every level of the component tree.

Normally, when data is passed from a parent component to a deeply nested child component, each intermediate component must receive and forward the props. This process is known as **Prop Drilling**.

The Context API solves this problem by creating a central context object that any component can access directly.

Advantages of Context API

- Eliminates prop drilling.
- Makes components easier to maintain.
- Simplifies data sharing.
- Improves code readability.
- Suitable for themes, authentication, language selection, and user preferences.

---

useContext Hook

The useContext() hook is used to consume the context value inside functional components.

Example:

const theme = useContext(ThemeContext);

Prerequisites

- Node.js
- npm
- Visual Studio Code
- ReactJS

Software Requirements

- React 17
- JavaScript (ES6)
- HTML
- CSS

Project Structure

employeesapp

```
├— src
│   ├— ThemeContext.js
│   ├— App.js
│   ├— Employee.js
│   ├— EmployeesList.js
│   ├— EmployeeCard.js
│   ├— EmployeeCard.module.css
│   ├— App.css
│   ├— index.js
```

```
|
├── package.json
├── package-lock.json
└── public
```

Implementation Steps

Step 1

Open the existing **employeesapp** project.

Step 2

Create a new file named

ThemeContext.js

Step 3

Create a React Context using

createContext()

Step 4

Wrap the application inside

<ThemeContext.Provider>

Step 5

Remove the theme prop from

EmployeesList

Step 6

Remove the theme prop from

EmployeeCard

Step 7

Use

useContext()

inside EmployeeCard.

Step 8

Run the project.

npm start

Source Code:

ThemeContext.js

```
import { createContext } from "react";
const ThemeContext = createContext();
export default ThemeContext;
```

App.js

```
import "./App.css";
import Employee from "./Employee";
import EmployeesList from "./EmployeesList";
import ThemeContext from "./ThemeContext";
function App() {
  const employees = [
    new Employee(
      1,
      "John",
      "john@gmail.com",
      "9876543210"
    ),
    new Employee(
      2,
      "David",
      "david@gmail.com",
      "9123456789"
```

```
    ),

    new Employee(
      3,
      "Peter",
      "peter@gmail.com",
      "9988776655"
    )

  ];

  const theme = "green";

  return (

    <ThemeContext.Provider value={theme}>

      <div className="App">

        <EmployeesList employees={employees} />

      </div>

    </ThemeContext.Provider>

  );

}
```

```
export default App;
```

Employee.js

```
class Employee {
  constructor(id, name, email, phone) {
    this.id = id;
    this.name = name;
    this.email = email;
    this.phone = phone;
  }
}
export default Employee;
```

EmployeesList.js

```
import EmployeeCard from "./EmployeeCard";
function EmployeesList(props) {
  return (
    <div>
      <h1>Employees List</h1>
      {
        props.employees.map(employee =>
          <EmployeeCard
            employee={employee}
            key={employee.id}
          />
        )
```

```
        }
      </div>
    );
}
export default EmployeesList;
```

EmployeeCard.js

```
import { useContext } from "react";
import Styles from "./EmployeeCard.module.css";
import ThemeContext from "./ThemeContext";
function EmployeeCard(props) {
  const theme = useContext(ThemeContext);
  return (
    <div className={Styles.Card}>
      <h3>{props.employee.name}</h3>
      <p>{props.employee.email}</p>
      <p>{props.employee.phone}</p>
      <p>
        <a href="#" className={theme}>
          Edit
        </a>
        {" | "}
        <a href="#" className={theme}>
          Delete
        </a>
      </p>
    </div>
```

```
  );
}
export default EmployeeCard;
```

EmployeeCard.module.css

```
.Card {
  display: inline-block;
  width: 25%;
  border: 1px solid #999;
  border-radius: 5px;
  margin: 10px;
  padding: 10px;
}
.green {
  color: green;
  font-weight: bold;
}
```

App.css

```
.App {
  text-align: center;
}
.green {
  color: green;
  text-decoration: none;
  margin-right: 15px;
}
```

---

index.js

```
import React from "react";
import ReactDOM from "react-dom";
import "./index.css";
import App from "./App";
ReactDOM.render(

  <React.StrictMode>
    <App />
  </React.StrictMode>,
  document.getElementById("root")
);
```

Output Screenshot:

When the application runs, the browser displays an **Employees List** page.

Each employee card contains:

- Employee Name
- Email Address
- Phone Number
- Edit link (Green)
- Delete link (Green)

Learning Outcome

After completing this exercise, I learned how to:

- Understand React Context API.
- Create a Context using createContext().
- Provide values using ThemeContext.Provider.
- Consume values using useContext().
- Eliminate Prop Drilling.
- Improve component communication.
- Build reusable React components.